Pneumonia Xray Classification

Problem

This assignment involves training a keras model to classify pneumonia patients based on chest ray images. The overall goal is to increase validation and testing accuracy as much as possible. Specifically, we will be focusing on the positive recall for sick patients. However, there are 3 classifications in this problem, normal, viral and bacterial, and both viral and bacterial would be considered positive when diagnosing a patient. Because of this, we will be focusing on increasing the recall of both bacterial and viral, with analysis including merged binary performance of normal and sick. This may be useful as in a real-world setting, as in any diagnosis classification algorithm, ensuring patients who are sick get treatment is the main priority. Further testing/diagnosis for the specific type of illness can come after. However, to keep in line with the project as specified, these classes will not be combined.

Data Exploration

This dataset has already been divided into 2 datasets, a training and testing set. The training dataset has 5419 images, and the test dataset has 438. The class distribution is as follows:

Dataset	Bacterial	Normal	Viral
Train	2596	1461	1362
Test	184	122	132

The dataset is moderately imbalanced, with bacterial being much more common in the training data than the other 2 classes. Experimentation with imbalance handling such as class weights could improve performance. This may include standard class weights, or custom class weights with combined weight for the bacterial and viral classes, focusing on the normal and sick merged binary performance.

The images are x ray images of patients with and without pneumonia. Since images are x rays, they will likely benefit from grey scaling during preprocessing to reduce the size to the dataset and remove unnecessary information.

A lack of domain knowledge in this area means that any further preparation, such as object extraction, will likely be unrealistic to implement in the given timeframe.

Baseline Model

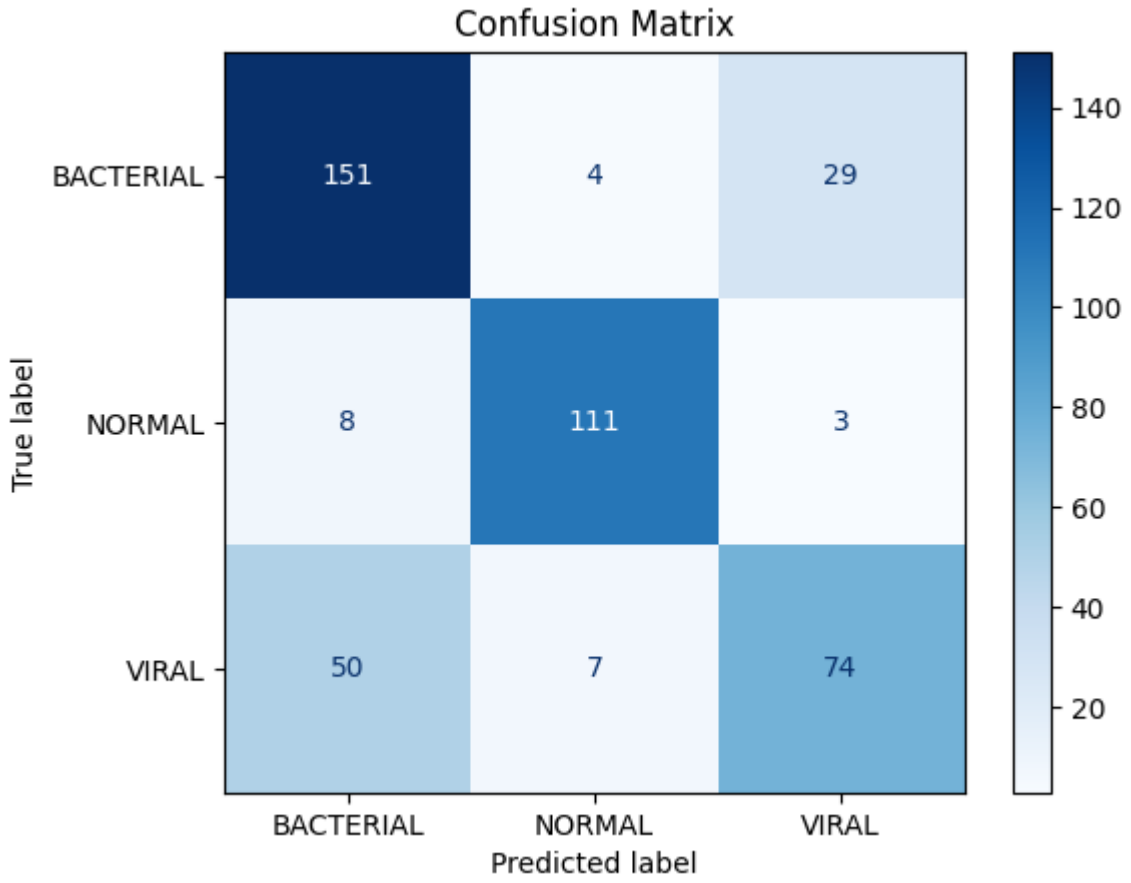
The baseline model performance taken here is based on the model provided in the python file for this assignment. This model has the following configuration.

1. Input of images size 128X128X3
2. Convolutional layer, 16 3X3 filters
3. Max Pooling layer, 2X2
4. Convolutional layer, 32 3X3 filters
5. Max Pooling layer, 2X2
6. Convolutional layer, 32 3X3 filters
7. Max Pooling layer, 2X2
8. Flattening
9. Dense layer, 512 nodes
10. Drop out, 20% of nodes dropped
11. Dense output layer, 3 class nodes

This model configuration ran for 20 epochs, using Adam as the loss function and accuracy as the scoring metric. The model with the best score was saved. Training with a GPU averaged about 8 seconds per epoch during training. Testing this model gave the following classification report:

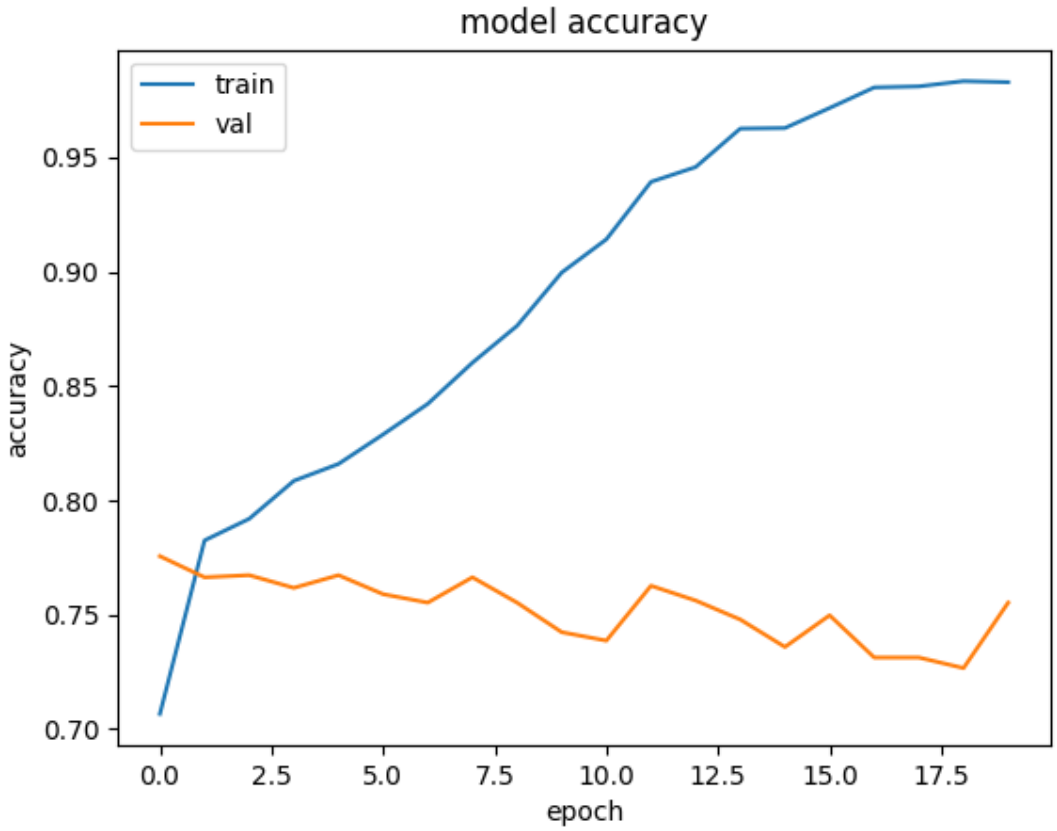
	precision	recall	f1-score	support
BACTERIAL	0.7081	0.8043	0.7532	184
NORMAL	0.8561	0.9754	0.9119	122
VIRAL	0.6742	0.4580	0.5455	131
accuracy			0.7483	437
macro avg	0.7461	0.7459	0.7368	437
weighted avg	0.7393	0.7483	0.7352	437

Overall, the performance here isn't great, with a testing accuracy of 74%. The main scoring metrics, the recall for the bacterial and viral classes are similarly poor, with the viral class only achieving a recall of 0.4580. Considering this is the least represented class in the dataset, this further shows that class imbalance handling will likely be beneficial. However, in this model, the normal class, despite not being the most represented, performed best. The confusion matrix for this model gives us a clear picture as to why.



The majority of the class confusion is between the 2 sick classes, casing them to both underperform compared to the normal class. Despite the poor performance, this is a good sign for predicting normal vs sick.

Looking at the model accuracy across the epochs, another issue in this implementation is overfitting.



With every epoch after the first overfitting on training data compared to validation. The most likely solution to this issue is to add data augmentation. Adding early stopping will also help not only to reduce overfitting but also in reducing training time.

Planned Experimentation

Baseline modelling and initial exploratory analysis have revealed several possible experiments which will likely improve performance.

Baseline results have shown that overfitting and class imbalance are 2 major concerns in the dataset. Adding data augmentations during training will likely reduce overfitting, as this often aids in model generalisation. Experimentation with class weights will likely improve accuracy in underperforming classes, specifically in the viral class.

As mentioned in data exploration, grey scaling will likely also be appropriate, since images are already in a black and white format.

Aside from these experiments, altering the model configuration may also improve performance. How exactly the model may be altered for better performance will become clearer after the initial experimentation has stabilised training. Transfer learning may also benefit performance, so experimentation with pretrained models such as Resnet and Efficientnet may prove useful, although these will need to be

altered to a new input structure if grey scaling is used, as these models are trained on colour images.

All further experiments would also likely benefit from early stopping, so this will also be used. Specifically, if the validation loss has stopped progressing after 5 epochs, training will end.

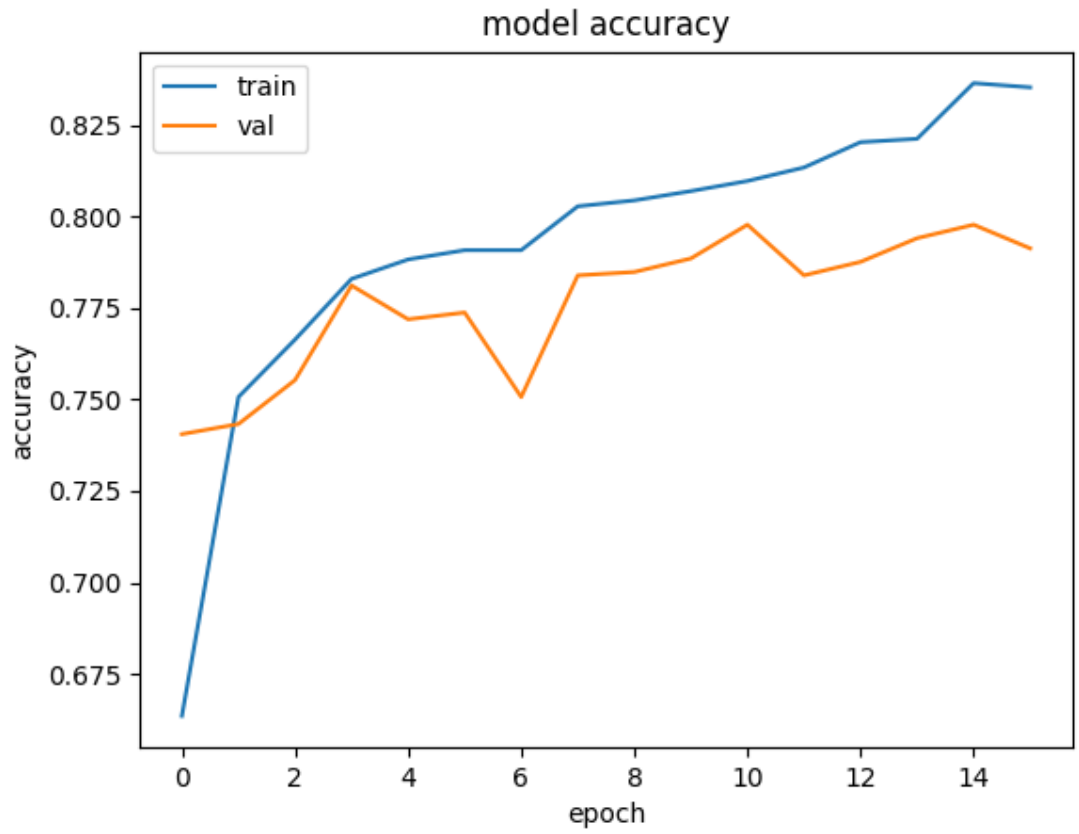
Augmentation and Preprocessing

Augmentation will likely be beneficial in improving model performance, specifically to reduce overfitting on training data (Shorten and Khoshgoftaar, 2019). Augmentations will only be applied to the training portion of the data, both validation and testing data will be kept as they are for constancy in model analysis. Possible augmentations which will likely improve performance are horizontal flips, slight rotations (<45 degrees) and slight zooming. These augmentations experiments were chosen as they will appropriately alter the images while preserving the information used for classification. Some other common augmentations where ignored, such as colour jitter, since this is redundant when grey scaling is used, and vertical flips, since altering the image orientation to this degree may alter image aspects related to classification. Horizontal flips were chosen as this should alter image without obscuring details used for classification. For this same reason, only minor rotations and zooming will be applied.

The augmentation experiments tested gave the following results.

Rotation	Zoom	Train Acc	Val Acc	Test Acc	Test F1
0.01	0.03	0.8425	0.7904	0.8046	0.8059
0.03	0.03	0.8088	0.7996	0.7963	0.7889
0.01	0.05	0.8097	0.7978	0. 8124	0.8105
0.03	0.05	0.8134	0.7950	0. 8009	0.7985

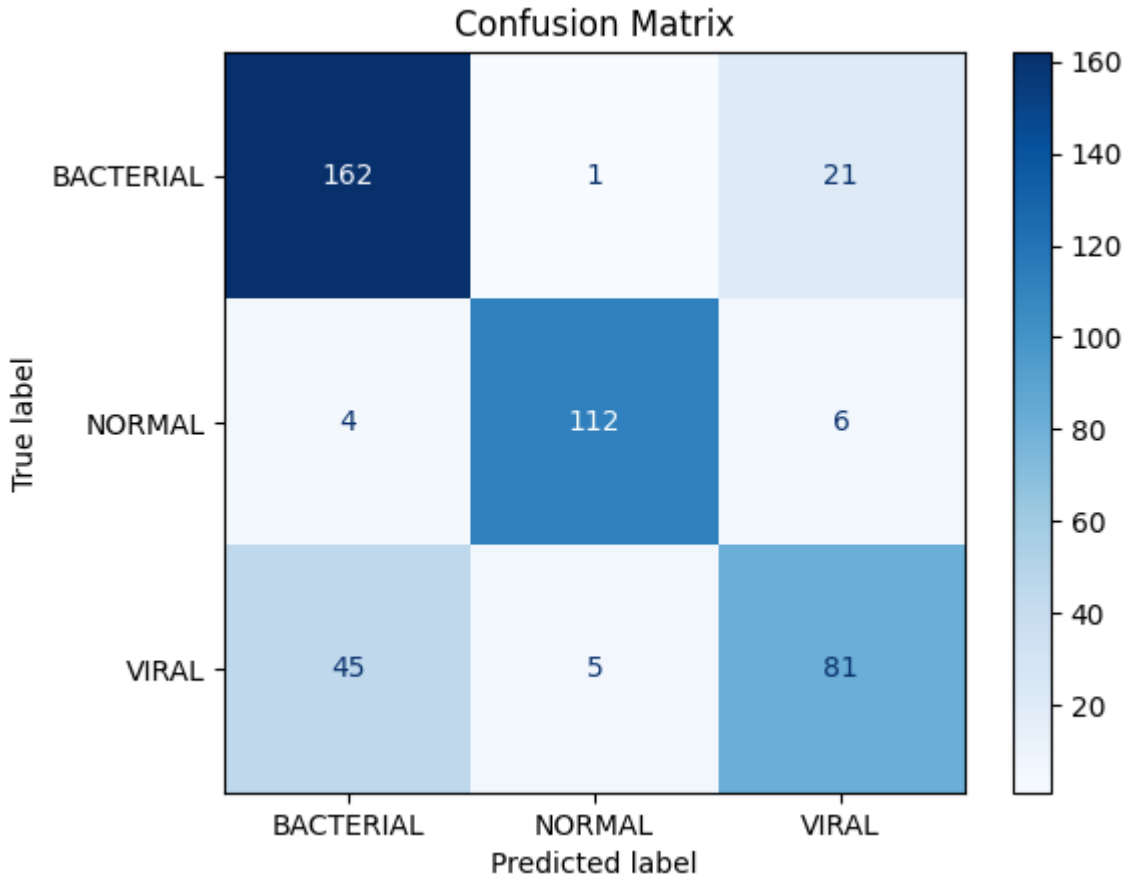
All of these implementations gave much better results compared to the baseline, with all of them except the first iteration having virtually no overfitting. The best implementation of augmentation had 0.01 rotation, meaning rotations of 1% were made randomly during training, and 0.03 zoom, meaning the image was zoomed in or out by up to 3% during training. These new images were generated each epoch, which increased the average training time to around 10 seconds per epoch.



The main benefit to this implementation was a massive reduction in overfitting during training. By making these random changes, providing the model with slightly altered new images each epoch allowed it to generalise better during training, leading to better scores across most metrics.

	precision	recall	f1-score	support
BACTERIAL	0.7678	0.8804	0.8203	184
NORMAL	0.9492	0.9180	0.9333	122
VIRAL	0.7500	0.6183	0.6778	131
accuracy			0.8124	437
macro avg	0.8223	0.8056	0.8105	437
weighted avg	0.8131	0.8124	0.8091	437

With bacterial and viral accuracies increasing significantly. However, this also led to a small decrease in normal recall. This implementation also greatly reduced the confusion between the viral and bacterial classes.



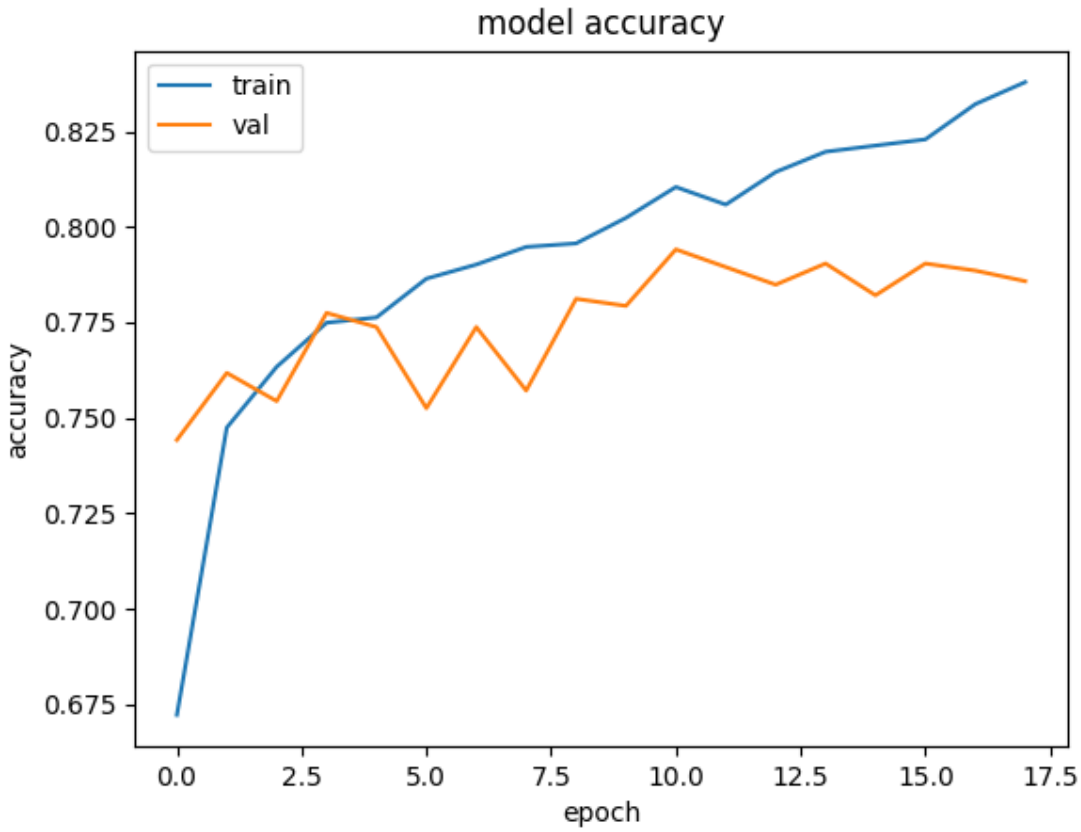
After augmentation, to reduce data size and remove unnecessary complexity, images were converted to grey scale during training. This also meant adjusting the input layer to take only 2 dimensions instead of 3. For this experiment to be successful, the same or better performance is preferred, since removing noise and reducing model complexity is beneficial even if performance stays the same. This change also reduced the training time needed to an average of around 8 seconds, likely since less transformations are needed.

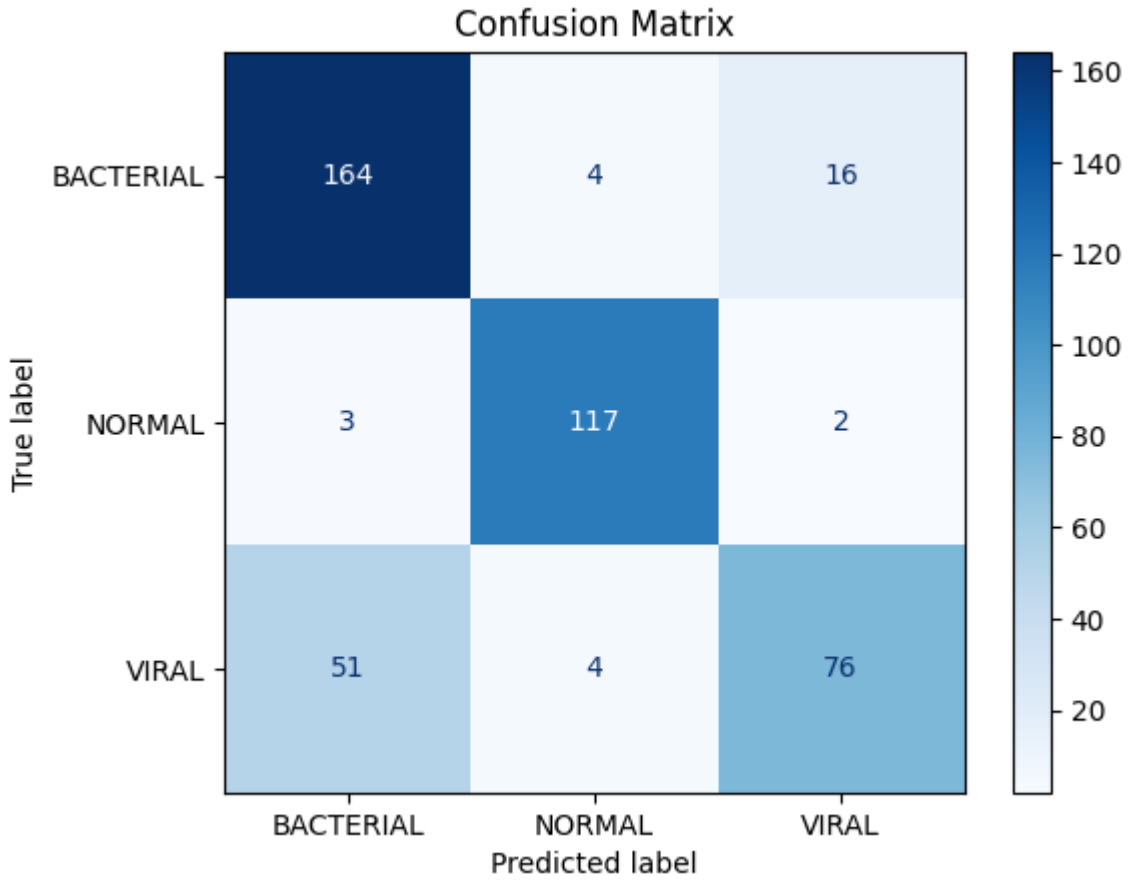
Overall this implementation performed similarly to the previous one, with slight shifts in accuracy across some metrics, with the main difference being an overall decrease in bacterial and increase in normal, both by 1 percent.

	precision	recall	f1-score	support
BACTERIAL	0.7523	0.8913	0.8159	184
NORMAL	0.9360	0.9590	0.9474	122
VIRAL	0.8085	0.5802	0.6756	131

accuracy			0.8169	437
macro avg	0.8323	0.8102	0.8129	437
weighted avg	0.8204	0.8169	0.8105	437

Training also went similarly in both implementations, minimal overfitting with a slight increase towards the end of training.





Confusion between the 2 sick classes, particularly with viral being misclassified as bacterial is still the primary issue, which the next experiments will hope to solve.

Class imbalance handling

The next experiments planned to improve performance is class imbalance handling. Even in the new implementation which has fixed issues with overfitting, the most underrepresented class, viral, is still underperforming compared to the others.

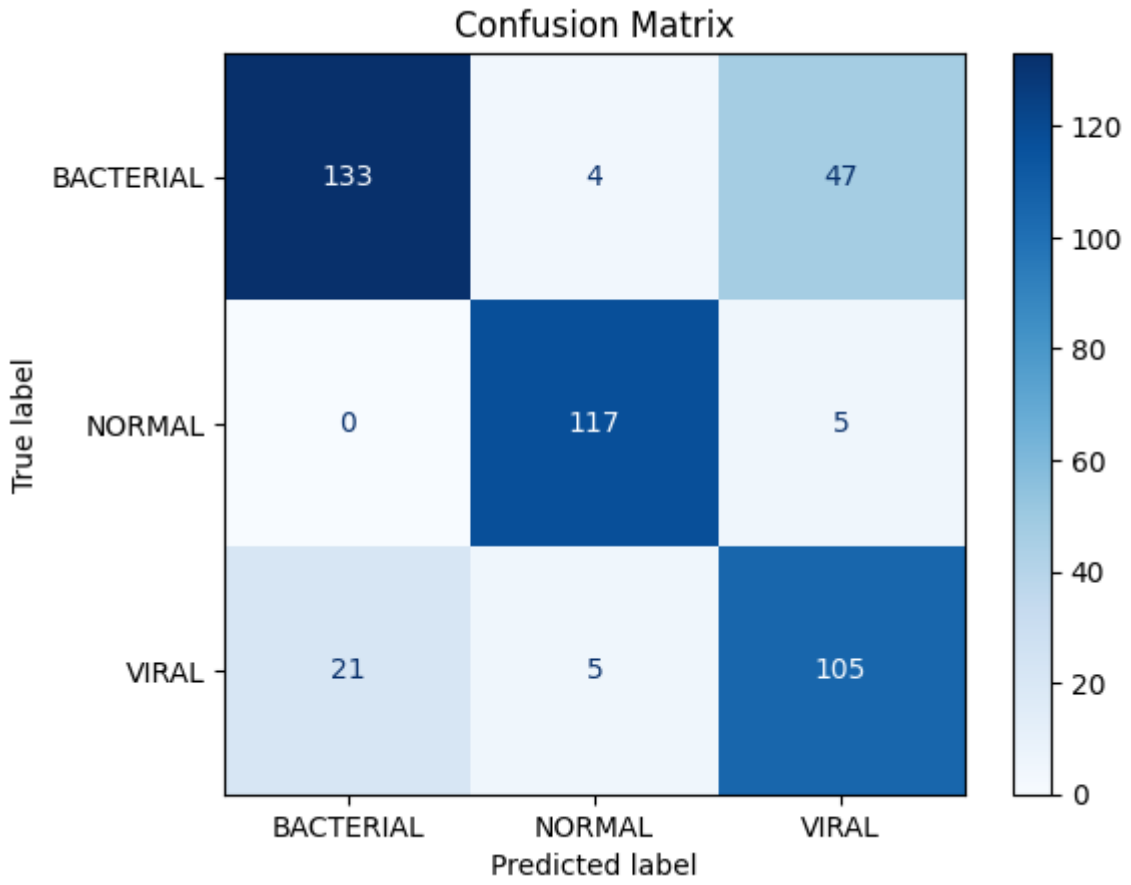
To address this issue, we will be experimenting with class weights. Using the class weights method, classes with higher class weights increase the effect those samples have on the loss function, resulting in greater changes in model weights for those classes during backpropagation (He and Garcia, 2009). Class weights were implemented using scikit learns compute class weights, which calculates weights with the following formula.

$$w_j = \frac{\text{Total number of samples}}{\text{Number of classes} \times \text{Number of samples in class } j}$$

With this calculation, classes with lower representation, like the viral class, which is underperforming, is given a higher-class weight.

This implementation had similar overall scores compared to the previous implementation, with the following classification report and confusion matrix.

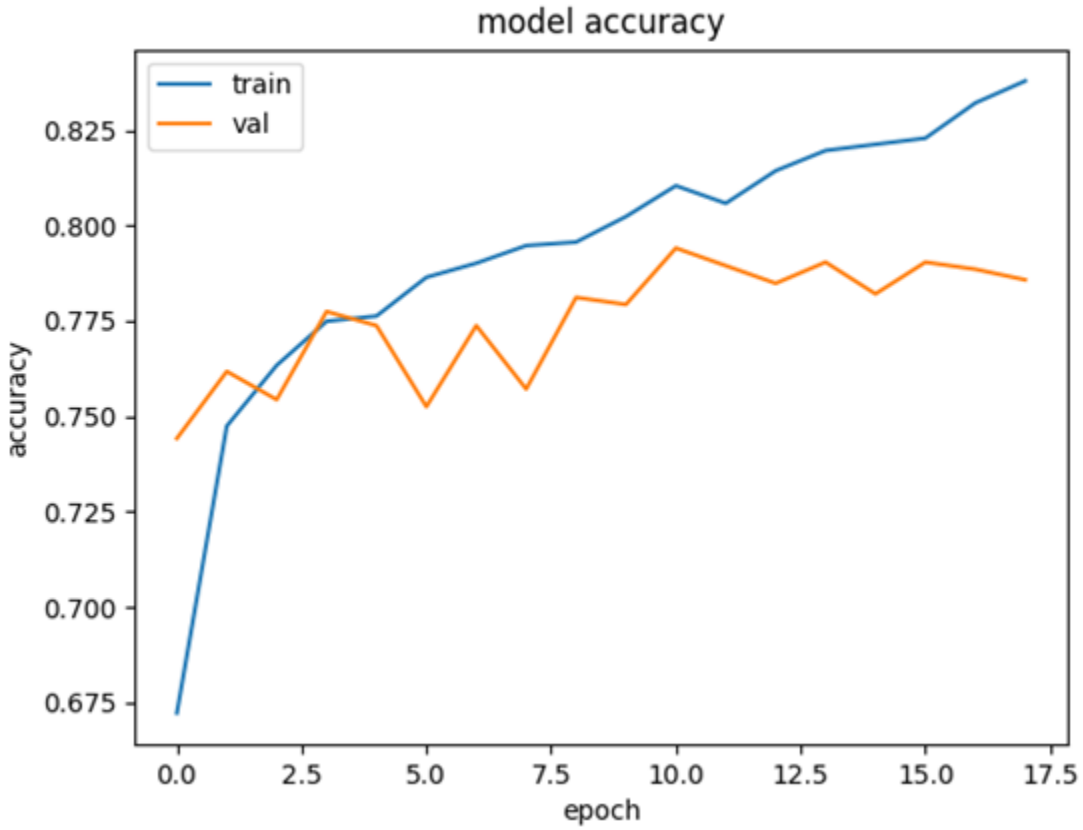
	precision	recall	f1-score	support
BACTERIAL	0.8636	0.7228	0.7870	184
NORMAL	0.9286	0.9590	0.9435	122
VIRAL	0.6688	0.8015	0.7292	131
accuracy			0.8124	437
macro avg	0.8203	0.8278	0.8199	437
weighted avg	0.8234	0.8124	0.8134	437



However, this implementation had much better results in the viral class, with an f1 increase of 5% compared to the previous implementation and a 12% increase in the

recall for this class. However, this did lead to a 3% decrease in the f1 score for the bacterial class, likely due to the fact that this had the lowest weight, since it was the most represented class. Since this implementation had higher overall recall and f1 compared to the last, we will continue with this implementation.

This implementation had similar training cycle to the previous, with minimal overfitting throughout and a slight increase at the end of training, not a concerning amount.



Model configurations

Now that issues around overfitting and class imbalance have been addressed, we can experiment on the model itself, with experiments including model structuring and regularisation.

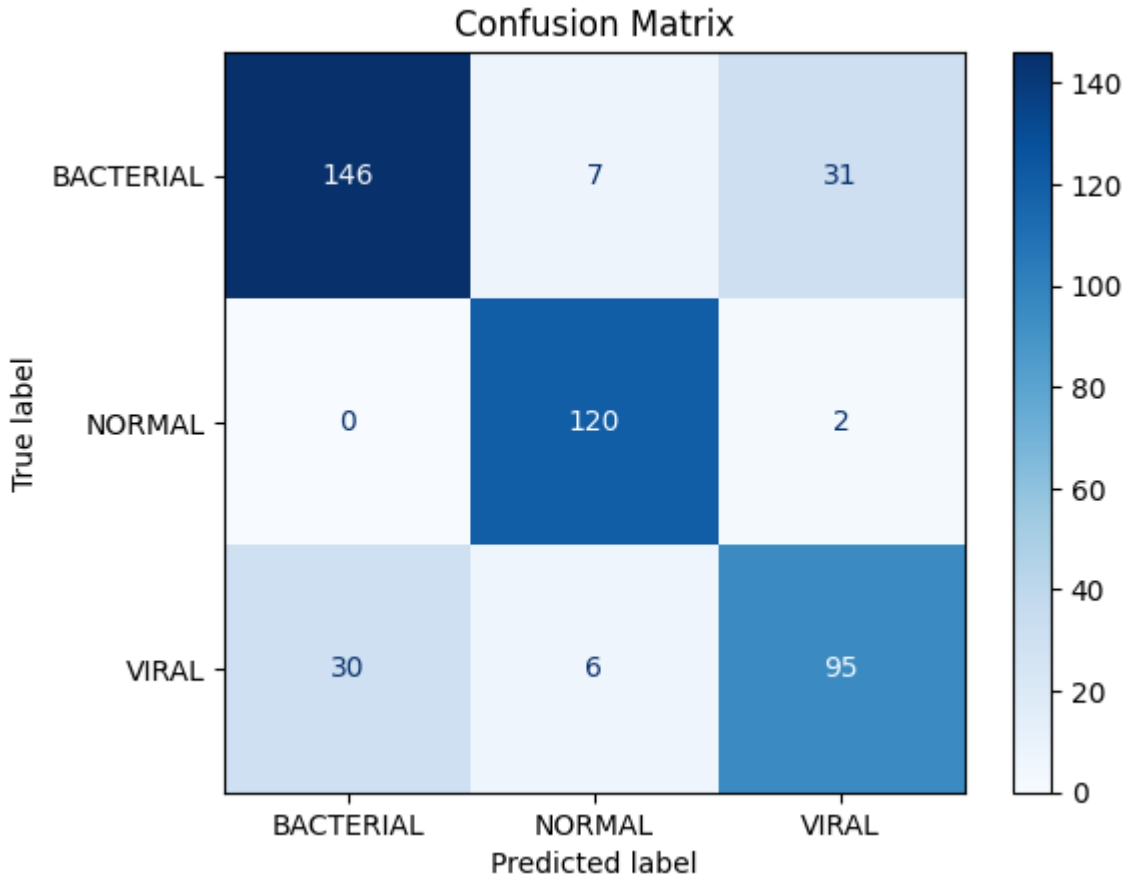
The first of these experiments will include adjustments in model size, specifically model upscaling to see if this results in improved performance. Initial experiments convolutional layers with 16, 32 and 32 layer, each followed by max pooling layers. Further experiments were done with the following structures.

Kernels per layer	Total kernels	Train acc	Val acc	Test acc	Test F1
32, 64, 64	160	0.8436	0.7913	0.8124	0. 8156
32, 64, 128	224	0.8310	0. 8006	0.8261	0. 8286

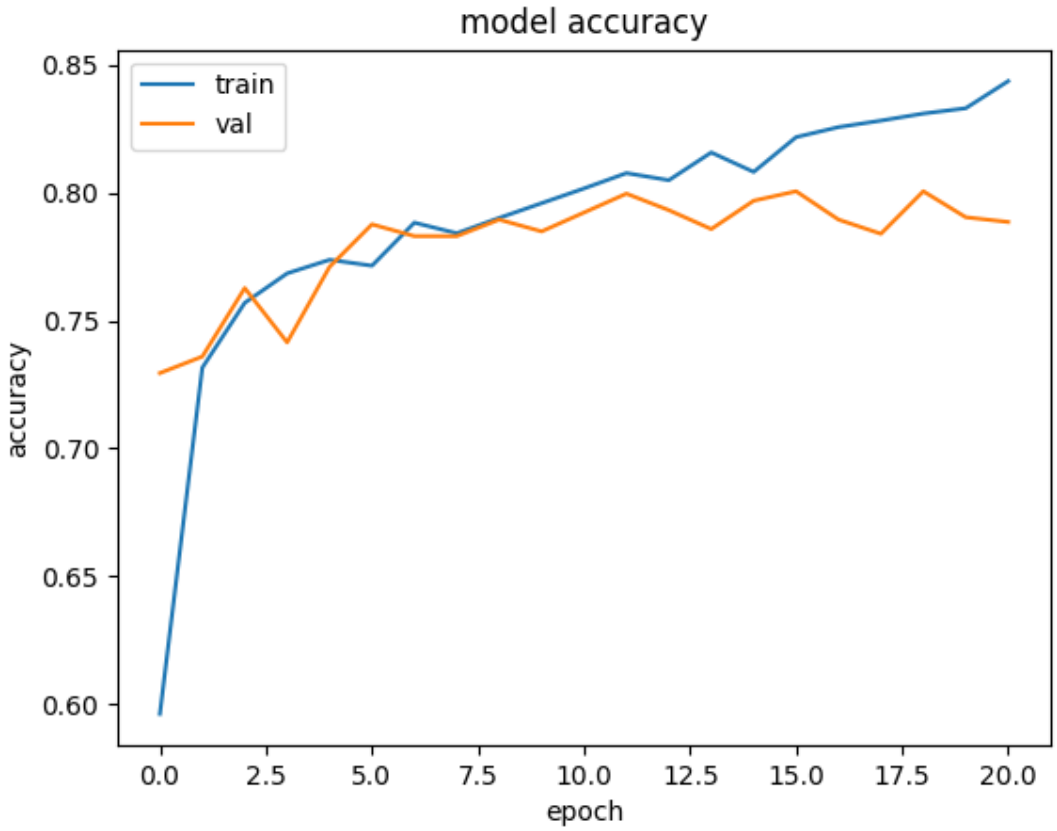
These model changes, specifically increasing the number of kernels at each layer, lead to increased performance. Greater model complexity has seemed to benefit model performance, particularly in the underperforming classes.

	precision	recall	f1-score	support
BACTERIAL	0.8295	0.7935	0.8111	184
NORMAL	0.9023	0.9836	0.9412	122
VIRAL	0.7422	0.7252	0.7336	131
accuracy			0.8261	437
macro avg	0.8247	0.8341	0.8286	437
weighted avg	0.8237	0.8261	0.8242	437

Performance gains were most noticeable in the bacterial class, with an F1 increase of 3%, and viral with an increase of 1%. Class confusion between the sick classes also dropped, primarily due to better recall scores in the bacterial class.



Training accuracy stayed consistent with previous experiments, with slightly less noise from the validation accuracy across training, as is expected with larger models. Training time only increased slightly in this implementation, at about 11 seconds per epoch. This may indicate that the main bottleneck on training speed is CPU related, since minimal change is seen with larger models, which would put more stain on GPU resources.



After these convolutional layers, the baseline model featured a flattening layer, followed by a dense layer with 512 neurons with dropout of 0.2 before the output nodes. Some other features that can often improve performance in CNN models are replacing flattening with global average pooling and adding batch normalisation in between convolutional layers and max pooling layers. Flattening simply involves converting the 2 dimensional outputs from the convolutional layers to a single dimension by reshaping the feature maps, all node values are kept in the process. Global average pooling also converts the 2 dimensional outputs to a single dimension, but rather than rearranging the 2 dimensional outputs to a single dimension it takes the average value of each feature map and makes them the values passed in the next layer. As this reduces the number of inputs to the next layer, experimentation with fewer neurons in the dense layer after it will also be experimented with. Batch normalisation is used to normalise the outputs at each layer during training by converting the outputs at each layer to have a mean of 0 and standard deviation of 1 using z score. This often increases model stability and accuracy.

The results of these experiments were as follows.

Experiment	Train Acc	Val acc	Train Acc	Train F1

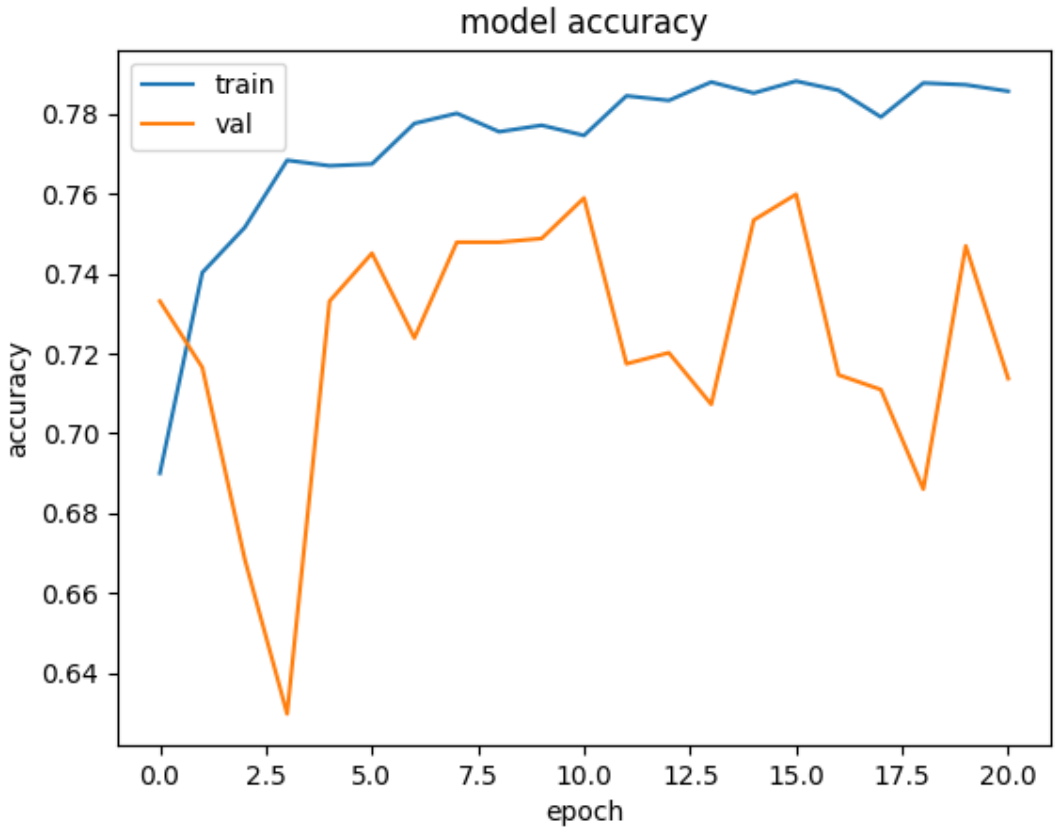
Global average pooling	0.7685	0.8052	0.7666	0.7573
256 Dense layer	0.7996	0.7922	0.7941	0.8017
1024 Dense layers	0.8374	0.7858	0.8055	0.8083
Batch Normalisation	0.7625	0.7784	0.7849	0.7774

All of these models underperformed compared to previous implementations. Global average pooling led to much slower learning rates, taking more epochs before performance levelled off, at which point it still underperformed compared to previous implementations. Both 256 and 1024 neurons implementations underperformed compared to 512 in the final dense layer, with 1024 beginning to overfit and 256 underfitting. Batch Normalisation also underperformed in both training and validation scores. None of these regularisation methods will be continued with.

Transfer learning

Now that the optimal model configuration has been found, experimentation with transfer learning can be conducted. Transfer learning involves using models which have been pretrained on large datasets to reuse learned features (Pan and Yang, 2010). Specifically, EfficientNetB0 will be used, since EfficientNet is commonly used for image classification and often outperforms resnet equivalents. EfficientNetB0 will be used as this is the smallest EfficientNet model, and since our model is considerable smaller compared to EfficientNet, the smallest of their models would be most appropriate. EfficientNet models are trained on colour images, and will expect a 3 channel input, so we will need to provide one. Rather than reintroducing the noise of the actual colour image, we can still take the greyscale image and simply passed this into all 3 colour channels. The results from this experiment were as follows.

	precision	recall	f1-score	support
BACTERIAL	0.8611	0.5054	0.6370	184
NORMAL	0.8992	0.9508	0.9243	122
VIRAL	0.5400	0.8244	0.6526	131
accuracy			0.7254	437
macro avg	0.7668	0.7602	0.7380	437
weighted avg	0.7755	0.7254	0.7219	437



For this problem, transfer learning using EfficientNetB0 underperformed compared to the custom model. The model had lower scores on training and validation, while also starting to overfit towards the end of training. This is likely due to the fact that this model is larger and more complex than the custom model. Transfer learning won't be continued with.

Final model

The final model used was a custom model, with an input size of 128X128, followed by 3 convolutional layer, max pooling layer pairs, with 32, 64 and 128 kernels each. These layers are followed by a dense layer with 512 neurons and dropout of 0.2, ending with the final output layer for classification. The model was trained using the Adam optimiser, with early stopping if there was no improvement in validation after 5 consecutive epochs. Class weights were used to account for class imbalances and augmentation was applied to images each epoch, including random horizontal flips, random rotations of up to 1% and random zooming of up to 5%. The model was trained on the greyscale versions of the images, scaled down to 128X128 for training.

Explainability

One of the main drawbacks of convolutional neural networks is the inherent lack of explainability with them. These models are black box, meaning unlike standard algorithms like object extraction and shape analysis, we don't know how the models are assessing images and why they classify as they do. However, we can gain a better understanding of how models make decisions using explainability tools, such as Grad-CAM. Grad-CAM is used to produce a heatmap over images, highlighting what areas were most important for classification the specific example (Selvaraju et al., 2017). The results of 3 Grad-CAM heatmaps are show below, once from each correctly predicted class.

Normal

This appears to be the most promising of the heat maps. The most indicative areas are all within the ribcage, specifically on the lungs. Interestingly, this image had very few significantly predictive areas, with activating regions only in specific spots.

Bacterial

Viral

Bacterial, as well as viral, weren't as promising as normal, with both having a lot of activation outside the ribcage. This may be the reason for misclassifications within these 2 classes, as the model is learning patterns not actually associated with Pneumonia classification. It could also indicate data leakage/noise within the dataset, if the sick images came from a different source than the normal images, this could lead to changes in the background leading to this inconsistency. However, both do still show activation within the rib cage, with both also having more activation areas within the rib

cage than the normal case, indicating that the model is likely learning some correct patterns.

References

He, H. and Garcia, E. (2009) Learning from Imbalanced Data. IEEE Transactions on Knowledge and Data Engineering.

Pan, S. and Yang, Q. (2010) A Survey on Transfer Learning. IEEE Transactions on Knowledge and Data Engineering.

Shorten, C. and Khoshgoftaar, T. (2019) Image Data Augmentation for Deep Learning. Journal of Big Data.

Selvaraju, R.R. et al. (2017) Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.

TensorFlow (2024) Keras Documentation. Available at: https://www.tensorflow.org/

Scikit-learn (2024) Machine Learning Library Documentation. Available at: https://scikit-learn.org/